

Cheat Sheet Machine Learning e Programmazione

Sezione 1: Python base

1.1 Indicizzazione e slicing (liste e NumPy)

Lo slicing permette di ritagliare porzioni di liste o array n-dimensionali. La sintassi base per matrici bidimensionali segue la struttura `M[riga, colonna]`.

- **Sintassi di riferimento:** `M[:, 7]`
- **Significato:** seleziona tutte le righe (con il simbolo `:` prima della virgola) e l'ottava colonna (indice 7, data indicizzazione a base 0).

```
L = [1, 2, 3, 4]
L[0]      # primo elemento
L[-1]     # ultimo elemento
L[1:4]    # elementi da indice 1 a 3 (4 escluso)
L[:3]     # elementi da indice 0 a 2
L[2:]     # elementi da indice 2 alla fine
L[::-1]   # lista invertita
```

funziona anche con array numpy.

```
import numpy as np
```

Immaginando una matrice M di dimensioni 10x10:

```
M[0, :]   # Estrae la prima riga completa (tutte le colonne)
```

```
M[:, 7]   # Estrae l'ottava colonna completa (tutte le righe)
```

```
M[2:5, 7] # Estrae le righe da indice 2 a 4 (l'estremo superiore 5 è ESCLUSO) dell'ottava colonna
```

1.2 Dizionari

Per iterare correttamente sugli elementi di un dizionario estraendo contemporaneamente la chiave e il valore associato è obbligatorio usare i metodi dedicati.

- **Sintassi di riferimento:** `dizionario.items()`
- **Errore comune:** l'iterazione diretta (`for chiave, valore in dizionario`) restituisce solo le chiavi, generando un `ValueError` per sbilanciamento in fase di assegnazione.

```
# APPROCCIO CORRETTO: utilizzo di .items()
for chiave, beta in optimal_parameters.items():
    # 'chiave' contiene la chiave del dizionario, 'beta' il valore associato
    pass
```

1.3 Assegnazione multipla e tuple unpacking

L'unpacking consente di distribuire gli elementi di un iterabile a variabili distinte in un'unica riga di codice. È un pattern fondamentale nel preprocessing per separare le feature dal target.

- **Sintassi di riferimento:** `var1, var2 = iterabile`
- **Significato:** assegna in ordine sequenziale i valori dell'iterabile alle variabili dichiarate a sinistra dell'uguale

```
# unpacking
a, b = [1, 2]
```

1.4 Aggregazione e risoluzione dei pareggi via Counter

La classe `Counter` del modulo `collections` mappa gli elementi di un iterabile come chiavi e le loro frequenze come valori.

- **Sintassi di riferimento:** `Counter(list).most_common(1)[0][0]`
- **Significato:** estrae l'etichetta di classe più frequente e risolve i pareggi.

```
from collections import Counter
```

```
# Inizializzazione del Counter su una lista di categorie
counter = Counter(categorie)
```

```
# Estrazione della classe a frequenza massima:
# counter.most_common(1)      -> Restituisce una lista con una tupla: [('category1', 2)]
# counter.most_common(1)[0]   -> Estrae la prima tupla: ('category1', 2)
# counter.most_common(1)[0][0] -> Estrae l'etichetta della classe: 'category1'
# counter.most_common(4)      -> Restituisce le 4 classi più comuni in ordine DECRESCENTE di frequenza
```

```
# NOTA COMPORTAMENTALE: In caso di pareggio tra frequenze, most_common() restituisce sempre la prima incontrata.
```

1.5 Ispezione dei tipi di dato (type e dtype)

Nei task di Machine Learning, è vitale assicurarsi che i tensori in ingresso ai modelli abbiano il corretto formato numerico.

- **Sintassi di riferimento:** `type(variabile)` o `oggetto.dtype`
- **Significato:**
 - `type(variabile)` restituisce il tipo della variabile
 - `oggetto.dtype` funziona solo per array numpy e series pandas e restituisce il tipo di elementi contenuti

```
# Ispezione del tipo di dato memorizzato nell'array
print(oggetto.dtype)
# Output di esempio: 'float64' o 'int64'.
```

```
# Più in generale, il tipo di un oggetto è ispezionabile con il builtin:
print(type(oggetto))
# Output di esempio: <class 'numpy.ndarray'>, <class 'pandas.core.frame.DataFrame'>
```

1.6 Rimozione di elementi da liste e dizionari

Python offre tre costrutti nativi per eliminare elementi. Operano tutti in modalità in-place (modificano direttamente la struttura originaria senza allocare nuova memoria).

- **Concetto chiave:** scegliere il metodo in base a ciò che si conosce (indice posizionale vs valore effettivo).
- **Significato:** `.pop()` usa l'indice e restituisce il valore, `.remove()` usa il valore, `del` usa l'indice o lo slicing.

#LISTE:

1. Metodo .pop(i): rimuove e restituisce l'elemento posizionato all'indice i
`elemento_rimosso = lista.pop(i)`

2. Metodo .remove(valore): individua ed elimina la PRIMA occorrenza del valore specificato
`lista.remove(valore)`

3. Istruzione del: elimina l'elemento o la fetta (slice) all'indice specificato
`del lista[i]`

#DIZIONARI:

1. Istruzione del: rimuove la coppia chiave-valore in modo diretto.
ATTENZIONE: Genera un KeyError se la chiave non è presente nel dizionario.
`del mio_dizionario['chiave_da_eliminare']`

2. Metodo .pop(chiave, default): restituisce il valore associato alla chiave.
La coppia chiave-valore viene quindi rimossa dal dizionario.
Fornendo un valore di default (es. None) si evita il crash qualora la chiave fosse assente.
`valore_estratto = mio_dizionario.pop('chiave_da_eliminare', None)`

3. Istruzione del: elimina l'elemento o la fetta (slice) all'indice specificato
`del lista[i]`

1.7 Valutazione logica di iterabili (all)

Funzione utile per controlli logici rapidi sui dati (es. validare vettori booleani).

- **Sintassi di riferimento:** `all(iterabile)`
- **Significato:** restituisce `True` se tutti gli elementi sono veri. Ottimizzato tramite short-circuit evaluation: l'analisi del vettore si interrompe al primo elemento `False` incontrato.

ESEMPIO: Verificare se una matrice presenta tutte colonne omogenee, ovvero se ciascuna colonna ha tutti gli elementi uguali

`import numpy as np`

```
A = np.array([[1, 2, 3],
              [1, 5, 3],
              [1, 8, 3]])
```

```
omogenee_colonne = all(len(np.unique(A[:, i])) == 1 for i in range(A.shape[1]))
```

UTILIZZO DI np.all:

```
pressioni = np.array([12.5, 14.2, 15.0, 9.8])
```

```
tutti_sicuri = np.all(pressioni > 5.0)      # True (tutti > 5.0)
tutti_alti = np.all(pressioni > 10.0)      # False (9.8 rompe la condizione)
```

1.8 Programmazione Orientata agli Oggetti (OOP)

metodi di istanza vs statici

- **Sintassi di riferimento:** @staticmethod vs def metodo(self):
- **Significato:** i metodi d'istanza (self) leggono/modificano lo stato dell'oggetto. I metodi statici non hanno accesso allo stato e si comportano come funzioni isolate dentro la classe.

```
class Test:

    # 0. Costruttore
    def __init__(self, argomento_dastampare:str):
        self.to_print = argomento_dastampare

    # 1. Metodo di Istanza: può usare attributi salvati nella classe
    def ciao(self):
        print("Ciao", self.to_print)

    def saluta(self):
        self.ciao() # Chiamata interna obbligatoria tramite self

    # 2. Metodo Statico
    @staticmethod
    def ciao_statico():
        print("ciao") # Non ha accesso a 'self'

# Utilizzo la classe
oggetto1 = Test("Mario")
oggetto1.saluta() # Stamperà "Ciao Mario"
Test.ciao_statico() # Invocazione formale tramite il nome della Classe
```

Ereditarietà

Erediti tutti i metodi della classe genitore e poi modifichi quelli che servono.

```
class TestPersonalizzato(Test):
```

1.9 Inizializzazioni di classi con dataclass

Una Dataclass è semplicemente una classe standard progettata principalmente per contenere dati, che utilizza il decoratore @dataclass per generare automaticamente codice boilerplate come `__init__` e `__repr__`.

- **Sintassi di riferimento:** @dataclass
- **Significato:** genera automaticamente i metodi standard basandosi solo sulle annotazioni dei tipi di attributi.

```
#CLASS NORMALE
class Test:
    def __init__(self, nome, eta, professione: str | None = None):
        self.nome = nome
```

```

        self.eta = eta
        self.professione = professione

#DATACLASS
from dataclasses import dataclass

@dataclass
class Persona:
    età: int
    nome: str
    professione: str | None

```

1.10 Parametri opzionali e valori di default nelle funzioni

Permette di specificare dei parametri di default nelle funzioni, senza che debbano essere specificati quando viene chiamata.

- **Sintassi di riferimento:** `def funzione(parametro=valore_default):`
- **Significato:** rende il passaggio dell'argomento facoltativo durante l'invocazione.

```

def funzione(x=10):
    pass

```

*# Se in fase di chiamata l'utente non specifica il parametro 'x',
verrà assegnato automaticamente il valore preimpostato (10).*

*# Attenzione: assegnare un oggetto mutabile (lista, dict) di default, ha l'effetto
controintuitivo di istanziare un oggetto in comune (globale) tra tutte le chiamate di
quella funzione, che condivideranno quell'oggetto mutabile, il quale rimane in memoria
durante il runtime.*

1.11 Gestione errori, eccezioni, type checking e debugging dimensionale (assert)

Nei task di Machine Learning, un disallineamento dimensionale tra tensori (ad esempio tra le label reali `y_true` e le previsioni `y_pred`) può causare errori di broadcasting silenziosi, producendo matrici con dimensioni sfalsate invece di interrompere l'esecuzione. L'uso di `assert` previene questo scenario (fail-fast) ed è altrettanto utile per validare i tipi di dato attesi.

- **Sintassi di riferimento:** `assert condizione, "messaggio di errore"`
- **Significato:** se la condizione è `False`, l'esecuzione si interrompe immediatamente con un `AssertionError` e il messaggio specificato. Esempio tipico: `assert y_true.shape == y_pred.shape, f"Shape mismatch: {y_true.shape} vs {y_pred.shape}"`.
- Per gestire eccezioni: `raise TipoDiEccezione("testo eccezione")`

--- IL COSTRUTTO ASSERT ---

Sintassi: assert condizione_da_verificare, "Messaggio di errore se False"

Se la condizione è True: non fa nulla e prosegue.

Se la condizione è False: interrompe l'esecuzione e lancia AssertionError.

```

assert y_true.ndim == 2, "y_true deve essere un vettore colonna 2D"

```

--- TYPE CHECKING (Validazione dei tipi di dato)

isinstance() verifica se un oggetto appartiene a una specifica classe (es. tuple, dict)

```

assert not isinstance(wine, tuple), "L'oggetto 'wine' non deve essere una tupla"

```

```

assert isinstance(ccps, dict), "La variabile non è un dizionario!"

```

```

# --- IL PERICOLO DEL BROADCASTING SILENZIOSO ---
# y_true = np.array([1, 0, 1])          # 1D, shape (3,)
# y_pred = np.array([[0.8], [0.2], [0.9]]) # 2D, shape (3, 1)

# (y_pred - y_true) applica il broadcasting!
# Invece di sottrarre elemento per elemento, crea una matrice errata di shape (3, 3).

# --- PERCHÉ USARE .ndim AL POSTO DI .shape ---
# L'attributo .ndim restituisce il numero di dimensioni (assi) di un array.
# shape (3,)    -> ndim = 1
# shape (3, 1)  -> ndim = 2
# shape (3, 1, 2) -> ndim = 3

# X APPROCCIO FRAGILE:
# assert y_true.shape[1] == 1 # Va in CRASH (IndexError) se y_true è 1D, fermando il
# codice nel modo sbagliato.

# V APPROCCIO ROBUSTO:
# assert y_true.ndim == 2      # Verifica in sicurezza che l'array sia effettivamente una
# matrice 2D.

# ECCEZIONI

```

Sezione 2: NumPy

2.1 Inizializzazione di array

```

indices = np.arange(a, b)          # crea un array con valori da a a b-1 con incremento
                                   # di 1 (es, np.arange(0,3) -> 0, 1, 2)

# np.ones richiede la shape passata come tupla (le doppie parentesi sono obbligatorie per
# matrici).
V = np.ones(5)                    # Vettore piatto 1D: [1., 1., 1., 1., 1.]
M = np.ones((3, 2))              # Matrice 3x2. Default dtype=float64

```

2.2 Dimensionalità, ridimensionamento e concatenazione

Il controllo delle dimensioni (shape e ndim) previene errori di broadcasting quanto si interfacciano array e modelli di machine learning.

- **Sintassi di riferimento:** `arr.shape` / `arr.ndim` / `arr.reshape(n, m)`
- **Significato:** `shape` restituisce una tupla con le dimensioni dell'array (es. (100, 8)).
`ndim` restituisce il numero di assi. `reshape` riorganizza i dati in una nuova forma senza copiare la memoria, a patto che il numero totale di elementi rimanga invariato.

```

# ---Attributi shape e ndim---
array_1d = np.array([1, 2, 3])      # shape (3,), ndim 1
array_riga_2d = np.array([[1, 2, 3]]) # shape (1,3), ndim 2
array_colonna_2d = np.array([[1], [2], [3]]) # shape (3,1), ndim 2
matrice_2d = np.array([[1,2,3],[4,5,6]]) # shape (2,3), ndim 2

# ---Cambio di dimensione---
matrice_flat = matrice_2d.flatten() # ritorna np.array([1, 2, 3, 4, 5, 6]), con shape (6,)
e ndim 1. funziona anche con array_riga_2d e array_colonna_2d.

```

```
array_1d_colonna = array_1d.reshape(-1,1) # forza un array 1D a diventare un vettore
colonna 2D (es. compatibilità  $X@beta$ ). Il valore -1 dice a NumPy di calcolare
automaticamente la dimensione mancante in base alla lunghezza. Per esempio questo avrà
shape (3,1).

# ---Concatenazione---
# Aggiunta di una colonna ("orizzontalmente") ad una matrice X come prima colonna
colonna = np.ones((X.shape[0], 1))
mat_concat = np.hstack((colonna, X))
# L'alternativa è np.concatenate, utile per esempio per unire due array 1D in uno.
concat_array = np.concatenate((array_1d,array_1d)) # ritorna np.array([1,2,3,1,2,3])
concat_X = np.concatenate([X[:2], X[6:]] # concatena le prime due righe di X con le righe
da indice 6 in poi, lungo asse 0 (righe)
```

2.3 Generazione di valori random

A differenza della libreria random standard di Python, le funzioni NumPy sono più efficienti e vettoriali.

- Sintassi di riferimento: `np.random.rand(n, m)` / `np.random.randint(low, high, size)` / `np.random.seed(42)` / `np.random.uniform(min, max)`
- Significato: `rand` genera una matrice $n \times m$ con valori float uniformi in $[0,1)$. `randint` genera interi casuali in un intervallo. `seed` garantisce che la sequenza random generata sia la stessa ad ogni esecuzione. `uniform(low,high)` genera dei valori casuali tra $[low, high)$. Low è compreso, high no.

```
import numpy as np

# --- NUMERI CASUALI E DISTRIBUZIONI ---
# randint(low, high): Interi pseudo-casuali. Il limite inferiore è incluso, il superiore è
ESCLUSO.
idx = np.random.randint(0, 10) # Singolo numero da 0 a 9
array_binario = np.random.randint(0, 2, size=(20,)) # Array 1D di venti elementi (solo 0 e
1)

# --- CAMPIONAMENTO CASUALE (Choice) ---
np.random.choice(a, size, replace, p)
# a : array da cui campionare (o intero  $n \rightarrow$  campiona da  $[0, n-1]$ )
# size : quanti campioni estrarre
# replace : True = con reinserimento, False = senza (ogni elemento estratto una sola volta)
# p : probabilità di ogni elemento (opzionale, default uniforme)

# PER LE MATRICI:
# --- FLOAT UNIFORMI IN  $[0, 1)$  ---
# Sintassi: np.random.rand(righe, colonne)
matrice_float = np.random.rand(3, 4) # Matrice 3x4 con float in  $[0, 1)$ 

# --- FLOAT UNIFORMI IN INTERVALLO PERSONALIZZATO ---
# Differenza chiave: rand genera solo in  $[0, 1)$ , uniform accetta bounds arbitrari
# np.random.rand(n, m)  $\rightarrow$  caso speciale di uniform con low=0, high=1
# per scalari (con lower_bound e upper_bound scalari) non serve specificare size per
ottenere un solo numero
np.random.uniform(low=lower_bound, high=upper_bound, size=(n_samples, n_features))

# --- DISTRIBUZIONE NORMALE ---
# distribuzione normale standard (media=0, std=1)
matrice = np.random.normal(size=(5, 5)) # Matrice quadrata 5x5

# Personalizzata: loc=media, scale=deviazione standard
matrice_custom = np.random.normal(loc=10.0, scale=2.0, size=(100, 3))
```

```
# --- SCELTA RANDOM TRA GLI ELEMENTI DI UN ARRAY ---
campione = np.random.choice(array, size=10, replace=False) # 10 elementi senza
reinserimento (cioè un elemento può essere scelto una volta sola)

campione_pesato = np.random.choice(array, size=5, replace=True, p=[0.1, 0.4, 0.3, 0.1,
0.1]) # assegna una probabilità ad ogni elemento dell'array

# NOTA: np.random.choice(n, size=k, replace=True) è equivalente a
# np.random.randint(0, n, size=k) quando si campiona da interi [0, n-1]

# --- PERMUTAZIONE RANDOM DI ELEMENTI (es. per indici) ---
permuted_array = np.random.permutation(array) # genera una permutazione casuale dell'array

shuffled_indices = np.random.permutation(n) # se n è un int, fa una permutazione
casuale di indici [0, 1, ..., n-1]. Di solito poi si usa con array_rimescolato =
array_originale[shuffled_indices]
```

2.4 Estrazione, indicizzazione e iterazione

Estrarre correttamente righe o singoli scalari permette di mantenere intatta l'integrità strutturale dell'array.

- **Sintassi di riferimento:** `arr[i]` / `arr[i, j]` / `arr[i, :]` / `arr[:, j]`
- **Significato:** `arr[i]` estrae la riga `i` come array 1D. `arr[i, j]` estrae lo scalare alla riga `i` e colonna `j`. `arr[i, :]` estrae la riga completa, `arr[:, j]` la colonna completa.

```
# --- ESTRAZIONE RIGHE E MANTENIMENTO SHAPE 2D ---
# Data una matrice X di shape (2,3):
riga_1d = X[0] # Estrazione standard: NumPy perde una dimensione, shape -> (3,)
riga_2d = X[0:1] # Estrazione via slicing: mantiene la matrice 2D, shape -> (1, 3)
riga_2d_alt = X[[0]] # Estrazione via fancy indexing: mantiene 2D, shape -> (1, 3)

# --- ESTRAZIONE SCALARE ---
# .item() estrae il valore puro Python da un array 1D singolo, ripulendolo dall'involucro
NumPy.
is_valid = a.item() >= 0.5 # Evita risposte del tipo array([True]), restituendo un puro
booleano True/False

# - ESTRAZIONE SCALARE SE CI SONO PIU' VALORI-
index = 5
is_valid = a.item(index) >= 0.5 # Se ci sono più valori, basta specificare l'indice del
valore che si vuole estrarre

# --- ITERAZIONE MATRICIALE ---
for row in X: # Itera sulle righe della matrice
    pass
for col in X.T: # Itera sulle colonne (iterando sulla matrice trasposta X.T)
    pass
```

2.5 Operatori logici, booleani e maschere

In NumPy, le normali keyword logiche di Python (`and`, `or`, `not`) sono sostituite da operatori bit-wise vettorializzati.

- **Sintassi di riferimento:** `&` / `|` / `~` / `arr[arr > soglia]`

- **Significato:** & (AND), | (OR), ~ (NOT) operano **elemento per elemento** su array booleani. L'indicizzazione booleana `arr[condizione]` restituisce solo gli elementi che soddisfano la maschera, senza loop espliciti.

```
# --- MASCHERE BOOLEANE (Filtering) ---
# Creazione maschera per splittare un dataset su una threshold.
right_mask = X[:, feature_idx] > threshold
X_right = X[right_mask, :] # Filtra le righe

# Negazione di una maschera.
left_mask = ~right_mask

# --- np.where (2 Modalità Operative) ---
# Modalità 1: Trovare gli indici (Ritorna una tupla di array, usiamo [0] per estrarre la
# lista indici).
indici_cat = np.where(y == 'cat')[0]

# Modalità 2: Sostituzione condizionale elemento per elemento.
y_sostituito = np.where(y > 0, 1, -1) # np.where(condizione, valore_se_True,
# valore_se_False)

# --- BINARIZZAZIONE TARGET ---
# Trasforma un array continuo in binario bilanciato usando la mediana come discriminante.
y_bilanciato = (y > np.median(y)).astype(int) # True/False convertiti a 1/0
```

Operatore	Nome Logico	Esempio Sintattico	Risultato Output
&	AND	True & False	False
	OR	True False	True
~	NOT (Negazione)	~True	False
^	XOR	True ^ True	False

2.6 Statistiche

- **Sintassi di riferimento:** `np.mean(arr)` / `np.std(arr)` / `np.sum(arr, axis=0)` / `np.sqrt(arr)`
- **Significato:** Calcolano rispettivamente media, deviazione standard, somma lungo un asse e radice quadrata. Il parametro `axis=0` opera **per colonna**, `axis=1` **per riga**.

```
# --- MASSIMI E INDICI (np.argmax) ---
# Restituisce l'indice del valore massimo. Il parametro 'axis' definisce la direzione di
# ricerca.
max_indici_col = np.argmax(A, axis=0) # Ricerca lungo le colonne (verticalmente)
max_indici_row = np.argmax(A, axis=1) # Ricerca lungo le righe (orizzontalmente)

# --- SOMMA CUMULATIVA (np.cumsum) ---
# Output: Ritorna array della stessa shape. L'array originale rimane invariato.
cum_flat = np.cumsum(array) # Senza asse (axis=None): appiattisce prima di
# sommare
cum_col = np.cumsum(matrice, axis=0) # Somma cumulativa lungo le colonne
cum_row = np.cumsum(matrice, axis=1) # Somma cumulativa lungo le righe

# --- VALIDAZIONE DELL'UGUAGLIANZA (np.allclose) ---
# Controlla se due matrici sono uguali entro una certa tolleranza numerica.
```

```
# Accetta solo ed esclusivamente 2 matrici (o array, o scalari, ma sempre 2).
sono_uguali = np.allclose(matrice_1, matrice_2)
```

2.7 Valori unici e ordinamento

La funzione `np.unique` è utile per trovare il set di categorie uniche in un array.

- **Sintassi di riferimento:** `np.unique(arr) / np.sort(arr)`
- **Significato:** `np.unique` restituisce gli elementi distinti in ordine crescente, eliminando i duplicati. Con `return_counts=True` restituisce anche la frequenza di ciascun valore.

```
# --- LABEL ENCODING NATIVO (return_inverse) ---
# Estrae i valori unici e converte simultaneamente i valori originali come indici numerici
(es. setosa->0).
classi_uniche, y_numeric = np.unique(y_text, return_inverse=True)

# --- CONTEGGIO FREQUENZE E ELEMENTO PIÙ FREQUENTE ---
# np.unique con return_counts=True ritorna i valori unici e le loro frequenze
classi_uniche, frequenze = np.unique(y, return_counts=True)
# np.argmax trova l'indice corrispondente al valore massimo, si usa poi per estrarre riga
della classe
indice_massimo = np.argmax(frequenze)
classe_maggioritaria = classi_uniche[indice_massimo]

# --- ORDINAMENTO INDICI ARRAY 1D E MATRICI ---
# np.argsort restituisce gli indici che permettono di ordinare gli elementi dell'array
lungo l'asse specificato. applicando tali indici all'array si ottiene la versione ordinata.
indici_ordinati = np.argsort(array_1d)      # es. se array è np.array([30, 10, 20]), indici
ordinati sarà np.array([1, 2, 0]).

indici_ordinati = np.argsort(matrice, axis=1) # se matrice è np.array([[30, 10, 20],[5, 15,
10]]) ordina gli elementi riga per riga,
indici_ordinati sarà np.array([[1, 2, 0], [0,
2, 1]]). Se axis=0, ordina colonna per
colonna.

# --- ORDINAMENTO ESTERNO (Tuple) ---
# Per ordinare una lista di tuple in al primo elemento di ciascuna tupla. Restituisce nuova
lista ordinata senza modificare quella originale.
coppie_ordinate = sorted(coppie_originali, key=lambda x: x[0])
```

2.8 Tabella delle conversioni inter-formato

Tabella riassuntiva dei metodi nativi per la transizione fluida tra i principali oggetti computazionali (NumPy, Pandas, liste native Python).

Origine \ Destinazione	In NumPy Array	In Pandas DataFrame/Series	In Python List
Da NumPy Array	-	<code>pd.DataFrame(array)</code>	<code>array.tolist()</code>
Da Pandas	<code>.to_numpy()</code>	-	<code>.tolist()</code> (su Series)
Da Python List	<code>np.array(lista)</code>	<code>pd.DataFrame(lista)</code>	-

alternativa a `.to_numpy()` è `.values`

2.8 Algebra lineare ed operatori

- **Sintassi di riferimento:** `np.eye(n) / np.linalg.inv(M) / A @ B / np.argmax(arr, axis=1) / arr[:, np.newaxis] / np.linalg.norm(array)`
- **Significato:** `eye` genera la matrice identità $n \times n$. `inv` calcola la matrice inversa. `@` è l'operatore di moltiplicazione matriciale. `argmin(axis=1)` restituisce l'indice del minimo per ogni riga. `np.newaxis` aggiunge una dimensione all'array (utile per broadcasting e distanze pairwise). `norm` calcola la norma (default euclidea)

```
# Norma di un vettore/matrice (Euclidea per default, ovvero L2)
np.linalg.norm(vettore)           # Norma dell'intero array come vettore piatto
np.linalg.norm(matrice, axis=1)    # Norma L2 di ogni RIGA (un valore per riga)
np.linalg.norm(matrice, axis=0)    # Norma L2 di ogni COLONNA (un valore per colonna)
```

Sezione 3: Pandas

3.1 Display, descrizione e statistiche

La seguente sintassi vale per DataFrame e Series pandas

- **Sintassi di riferimento:** `df.head()` / `df.info()` / `df.describe()` / `df['colonna'].value_counts()`
- **Significato:**
 - `head()` mostra le prime 5 righe.
 - `info()` rivela i dtype di ogni colonna e il conteggio dei NaN.
 - `describe()` calcola media, deviazione standard e percentili per le colonne numeriche.

```
# 1. Visualizzazione geometrica: mostra le prime 5 righe del DataFrame
df.head()
```

```
# 2. Ispezione strutturale: mostra informazioni su tipi di dato (dtype), righe totali e NaN
df.info()
```

```
# 3. Analisi statistica: calcola metriche descrittive (media, std, percentili) delle
colonne numeriche
df.describe()
```

```
# 4. Statistiche
median = df['NomeColonna'].median()
mean = df['NomeColonna'].mean()
mode = df['NomeColonna'].mode()[0]
pearson_corr = df.corr()
```

```
# raggruppa il dataframe in base ai valori della colonna 'Colonna1' e calcola la media e i
valori della colonna 'Colonna2' per ciascun gruppo.
df.groupby('col_name')['target_name'].mean()
```

3.2 Gestioni valori mancanti

- **Sintassi di riferimento:** `df.isnull().sum()` / `df.dropna()` / `df.fillna(valore)`

- **Significato:** `isnull().sum()` conta i NaN per colonna. `dropna()` rimuove le righe che contengono almeno un NaN. `fillna()` sostituisce i NaN con un valore specificato.

```
# --- CONTEGGIO E RIMOZIONE ---
df.isna().sum()          # Conta il numero totale di NaN per ogni colonna
df.dropna(inplace=True)  # Rimuove l'intera riga se contiene almeno un NaN

# --- IMPUTAZIONE DEI VALORI MANCANTI ---
# fillna(...) sostituisce i valori mancanti con valore specificato value, che può essere scalare (numero, stringa, booleano ecc) oppure una struttura series.
df['Age'] = df['Age'].fillna(value)
df['Age'].fillna(value, inplace=True) # per modificare in place, non ritorna nulla.
```

3.3 Conteggio e valori unici

L'identificazione dei valori unici in una colonna di un dataframe, in una series o in un np.array si effettua tramite la funzione `unique()`, ma il comportamento varia a seconda della libreria utilizzata.

- **Sintassi di riferimento:** `df['colonna'].unique()` vs `np.unique(arr)`, `df['colonna'].nunique()`, `df['colonna'].value_counts()`
- **Significato:** `pd.Series.unique()` restituisce i valori distinti nell'**ordine di prima apparizione** nel dataset. `np.unique()` restituisce i valori distinti in **ordine crescente**. `pd.Series.nunique()` ritorna il numero i valori distinti in quella colonna. `pd.Series.value_counts()` ritorna per ogni valore distinto il conteggio

```
# --- CONTEGGIO con pandas ---
# conta quanti elementi per ogni valore unico sono presenti in una colonna di un dataframe
df['ocean_proximity'].value_counts()

# conta il numero di valori unici
df['ocean_proximity'].nunique()

# --- VALORI UNICI: METODO PANDAS (.unique) ---
# Restituisce i valori distinti mantenendo l'ORDINE DI APPARIZIONE originale.
classi_pandas = data['species'].unique()

# --- VALORI UNICI: FUNZIONE NUMPY (np.unique) ---
# Restituisce i valori distinti forzando un ORDINAMENTO ALFABETICO/NUMERICO.
classi_numpy = np.unique(array_numpy)
```

3.4 Encoding

Per convertire valori testuali in valori numerici.

- **Sintassi di riferimento:**
 - Label encoding `df['colonna'] = df['colonna'].map({'A': 0, 'B': 1}), df['col'].astype('category').cat.codes`
 - One-hot encoding `pd.get_dummies(df, columns = ['colonna1', 'colonna2'], drop_first=True)`,
 - Label-encoding vettorizzato con `np.unque i(allineamento target-predizioni):` utilizzato quando il vettore delle predizioni `y_pred` viene generato in formato

stringa/objects (es. dall'algoritmo custom) ma deve essere confrontato con `y_test` che è in formato intero (int64).

- **Significato:** `.map()` applica una funzione o un dizionario **elemento per elemento** sulla Serie. Ogni stringa viene sostituita dal valore numerico corrispondente nel dizionario. Valori non presenti nel dizionario vengono convertiti in NaN. Alternativa a `map` è `df['col'].astype('category').cat.codes`.
`pd.get_dummies(df, columns = ['colonna'], drop_first=True)` crea nuove colonne binarie per ogni categoria presente nella colonna = 'colonna' nel dataframe e rimuove le colonne originali specificate. Se `drop_first = True` ne elimina una delle colonne generate.
- Nota: oppure si può usare la classe **Imputer** di sklearn (vedere cheatsheet base)

```
# Map esplicito di variabili testuali in interi (es. mappa tutti i valori = 'male' in 1)
X['Sex'] = X['Sex'].map({'male': 1, 'female': 0})
X['Embarked'] = X['Embarked'].map({'C': 0, 'Q': 1, 'S': 2})

# One-hot encoding: per le colonne 'Sex' e 'Embarked' in df, ed elimina le originali
df_encoded = pd.get_dummies(df, columns=['Sex', 'Embarked'])

# Converti le stringhe di y_pred in indici interi sequenziali (0, 1, 2...)
# return_inverse=True restituisce la posizione di ogni elemento originario rispetto alle
# classi uniche trovate
_, y_pred = np.unique(y_pred, return_inverse=True)
```

3.5 Discretizzazione e binning (bucketing)

Il partizionamento di una feature continua in classi categoriche ordinali è essenziale in contesti come il condition monitoring (es. trasformare un segnale continuo di temperatura in stati discreti di allarme).

- **Sintassi di riferimento:** `pd.cut(df['colonna'], bins=[...], labels=[...])`
- **Significato:** `pd.cut` suddivide i valori continui in intervalli definiti da `bins` e assegna a ciascun intervallo un'etichetta da `labels`. Gli intervalli sono **chiusi a destra** per default (es. (10, 20]). Utile per creare feature ordinali da segnali fisici.

```
import pandas as pd

temperature = [12, 18, 22, 27, 35]

# Trasforma i valori continui in classi: 0 (freddo), 1 (mite), 2 (caldo)
classi = pd.cut(
    temperature,
    bins=[0, 15, 25, 40], # Definisce gli scaglioni di taglio
    labels=[0, 1, 2],      # Etichette assegnate ai vari bin
    include_lowest=True    # Include il limite inferiore nel primo bin
)
```

3.6 Eliminare colonne e ricavare nomi colonne

- **Sintassi di riferimento:** `df.drop(columns=['col'])`
- **Significato:** `drop(columns=[...])` elimina le colonne specificate senza modificare il DataFrame originale (a meno di `inplace=True`).

```
df.drop(columns=['col1', 'col2']) # Rimuove le colonne specificate
feature_names = df.columns[:-1]  # prende tutti i nomi delle colonne tranne ultimo
```

3.7 Trovare feature categoriche

Per distinguere in modo semi-automatico le feature categoriche da quelle numeriche, si può contare il numero di elementi distinti presenti in quella feature/colonna. Se sono minori di una threshold n decisa da noi allora la trattiamo da categorica. Nota: è un'euristica, non una regola generale. La decisione va motivata anche conoscendo il significato della feature (come da descrizione del dataset).

```
#--- Separazione tra feature numeriche e categoriche ---
# Qui sotto 'k' è il nome della feature/colonna, e 'v' contiene il numero di valori
# distinti per quella feature, 'n' è l'intero scelto da noi.
# Stiamo creando un dizionario con chiavi il nome di ogni feature/colonna 'k' e con valore
# booleano, True se il numero di valori distinti in quella colonna era <= n e False
# altrimenti.
is_categorical = {
    k: v <= n for k, v in df.nunique().to_dict().items()
}

numerical_features = [k for k, v in is_categorical.items() if not v] # crea una lista con
# il NOME delle colonne che hanno valore False in nel dizionario is_categorical.
categorical_features = [k for k, v in is_categorical.items() if v] # crea una lista con il
# NOME delle colonne che hanno valore True in nel dizionario is_categorical.

# Converte i nomi delle colonne numeriche nei rispettivi indici all'interno del dataframe
X_df
numerical_indices = [X_df.columns.get_loc(col) for col in numerical_features]

# In alternativa si può usare pd.Series.nunique() (vedi sezione 3.3)

# Un'altra alternativa è usare df.select_dtypes(include=['object']).columns.tolist() che
# torna una lista dei nomi delle colonne contenenti il tipo di dato specificato.
⚠️❌ non robusto (SCONSIGLIATO): attenzione che a volte anche valori numerici possono
# essere codificati come oggetti.
df.select_dtypes(include=['object']).columns.tolist()
```

Errori comuni di sintassi (Pandas, NumPy, liste)

Alcuni costrutti e keyword non richiedono le parentesi tonde, poiché sono **proprietà** dell'oggetto o costanti, non metodi invocabili.

- **Sintassi di riferimento:** `df.shape` / `arr.ndim` / `True` / `False` / `None`
- **Errore comune:** Scrivere `df.shape()` o `arr.ndim()` genera un `TypeError`: `'tuple' object is not callable`. Le proprietà si leggono senza parentesi, i metodi si invocano con le parentesi.

```
# Proprietà e Costanti: NON usare le parentesi ()
valore_infinito = np.inf          # Costante matematica per infinito
dimensioni = X.shape              # Proprietà strutturale (tuple), non un metodo
```

```
# Iterazione su liste di tuple/combinazioni (Evitare enumerate se non serve l'indice)
# SBAGLIATO (genera errore di unpacking): for c1, c2 in enumerate(lista_combinazioni):
# CORRETTO (l'elemento estratto è già la coppia):
for classe1, classe2 in lista_combinazioni:
    pass

# Se c'è bisogno di avere anche gli indici delle tuple:
for indice, (classe1, classe2) in enumerate(lista_combinazioni):
    pass
```

Sezione 4: visualizzazioni con Matplotlib e Seaborn

4.1 Struttura spaziale: figure e subplots

La corretta istanziazione dell'area di disegno è fondamentale per affiancare metriche multiple. La Figure è la tela principale, mentre i Subplot sono le singole celle della griglia.

- **Sintassi di riferimento:** `fig, axes = plt.subplots(nrows, ncols, figsize=(w, h))`
- **Significato:** Crea una figura con una griglia di `nrows × ncols` assi. `axes` è un array NumPy di oggetti Axes — si accede al singolo pannello con `axes[i]` (1D) o `axes[i, j]` (2D). `figsize` definisce le dimensioni in pollici.

⚠ **Due stili a confronto:** Matplotlib ha due API.

```
# STILE OBJECT-ORIENTED (consigliato)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].plot(x, y)          # accesso esplicito al subplot
axes[1].scatter(x, y)

# STILE PYPLOT / STATEFUL
plt.figure(figsize=(15, 8))
for i in range(4):
    plt.subplot(1, 4, i + 1) # indice parte da 1, non da 0
    plt.plot(x, y)           # Matplotlib decide implicitamente quale subplot è attivo
```

4.2 Curve e distribuzioni

Visualizzazione per analizzare andamento generale di una variabile rispetto a un asse continuo e per studiare la distribuzione statistica di più gruppi di dati.

- **Sintassi di riferimento:** `ax.plot(x, y, label='...') / ax.boxplot(data) / ax.legend() / ax.set_xlabel('...') / ax.set_ylabel('...')`
- **Significato:** `plot` traccia una curva di valori `y` rispetto a una variabile continua `x`. `boxplot` visualizza la distribuzione statistica (mediana, IQR, outlier). `legend()` attiva la legenda basandosi sui label assegnati. `set_xlabel`, `set_ylabel` definiscono etichette assi.

```
import matplotlib.pyplot as plt
```

```

# CURVA (andamento di y in funzione di x)
plt.plot(x_values, y_values)
plt.xlabel('Parametro')
plt.ylabel('Valore')
plt.title('Andamento di Y')
plt.grid()
plt.show()

# BOXPLOT (per distribuzioni multiple confrontate tra gruppi)
plt.figure()
plt.boxplot(data_groups, tick_labels=labels, showmeans=True)
plt.ylim(bottom=0) # forza asse Y a partire da 0
plt.xlabel('Gruppi')
plt.ylabel('Valori osservati')
plt.title('Distribuzione dei valori per gruppo')
plt.show()

```

4.3 Visualizzazione titoli dinamici

La parametrizzazione dinamica dei titoli permette di riciclare il codice di plotting in loop su feature o configurazioni multiple.

- **Sintassi di riferimento:** `ax.set_title(f'Segnale: {nome_variabile}') / ax.plot(t, segnale)`
- **Significato:** Le f-string dentro `plt.title()` oppure `ax.set_title()` consentono di iniettare il nome della feature o del sensore corrente direttamente nel titolo del grafico. Indispensabile quando si visualizzano serie temporali in loop su canali multipli.

```

# --- RENDER DI IMMAGINI (Array 2D) ---
# Utile per task di Computer Vision. cmap='Greys' forza la scala di grigi
plt.imshow(array_2d, cmap='Greys')

# --- TITOLI DINAMICI ---
# Utilizzo f-string per inserire il valore di una variabile nel titolo
plt.title(f'Testo {variabile}')

```

4.4 Heatmap

- **Sintassi di riferimento:** `sns.heatmap(cm, annot=True, fmt='.2f', xticklabels=classi, yticklabels=classi)`
`sns.heatmap(matrix, annot=True, fmt='.2f', xticklabels=labels, yticklabels=labels)`
- **Significato:** `annot=True` stampa il valore numerico in ogni cella. `fmt='.2f'` formatta i valori come float a due decimali. `xticklabels` rappresenta le etichette dell'asse orizzontale, e `yticklabels` le etichette dell'asse verticale.

```

import seaborn as sns
import matplotlib.pyplot as plt

```



```
# matrice di valori già calcolata altrove
matrix = ...
```

```
plt.figure(figsize=(10, 8))
sns.heatmap(matrix, annot=True, fmt='.2f')
plt.xlabel('Colonne')
plt.ylabel('Righe')
plt.title('Heatmap della matrice')
plt.show()
```

4.4 Bar plot, istogrammi, boxplot e scatterplot

- **Sintassi di riferimento:** `ax.bar(etichette, conteggi) / ax.hist(errori, bins=n)`
- **Significato:** `bar` visualizza conteggi discreti per categoria. `hist` raggruppa valori continui in `bins` intervalli e conta gli elementi in quell'intervallo.

```
# --- BAR PLOT DA DIZIONARIO ---
# Estrae chiavi (asse X) e valori (asse Y) da un dizionario
dizionario = {'A': 12, 'B': 7.3, 'C': 5}
plt.bar(dizionario.keys(), dizionario.values())
plt.show()

# --- ISTOGRAMMA ---
# funziona con liste, array numpy e Series pandas
lista = [0.2, 0.5, 0.7, 1.2, 1.5, 1.8, 2.0, 2.1, 2.3, 2.8, 3.0, 3.2, 3.5, 3.7, 4.0, 4.5]
plt.hist(lista, bins=5)
plt.xlabel('Valori')
plt.ylabel('Conteggi')
plt.title('Istogramma dei Dati')
plt.show()

# --- BOXPLOT ---
# funziona con liste, array numpy e Series pandas, tick_labels assegna un nome ad ogni box plot
lista1= [1, 2, 3]
lista2= [2, 3, 4]
plt.figure()
plt.boxplot([lista1, lista2], tick_labels=['lista1', 'lista2'])
plt.xlabel("liste")
plt.title("...")
plt.show()

# --- SCATTERPLOT ---
# variabile1, variabile2 sono liste o array di valori numerici, c=labels assegna un colore diverso ad ogni unique value all'interno di "labels", s=50 da lo spessore ad ogni punto
plt.scatter(variabile1, variabile2, c=labels, s=50)
plt.title("...")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

4.5 Seaborn: visualizzazioni avanzate

- **Sintassi di riferimento:** `sns.pairplot(df) / sns.countplot(data=df, x=col1, hue=col2) / sns.violinplot(x=..., y=..., data=df, ax=axes[i])`

- **Significato:** pairplot crea griglie di grafici a dispersione tra tutte le colonne numeriche del dataframe. countplot (con hue) conta le occorrenze di una colonna `x = col1` e le suddivide in base alla variabile/colonna `col2` definita su `hue=col2`. violinplot combina boxplot e distribuzione.
- **Sintassi di riferimento:** `axes.flatten()` / `plt.xticks(rotation=45)`
- **Significato:** `flatten()` converte una griglia 2D di axes in un array 1D — indispensabile per iterare i subplots in un loop. `rotation=45` ruota le etichette dell'asse x per evitare sovrapposizioni.

Sezione 5: deep learning (Tensorflow, Keras)

5.1 Istanziare un modello, training e evaluation

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input, Conv2D, Dropout, MaxPooling2D

import tensorflow as tf

model = Sequential()

# Layer types that can be added
model.add(Dense(n_nodes, activation='...'))
model.add(Input(shape=...))
model.add(Flatten())

model.add(MaxPooling2D(pool_size=(...,...)))
model.add(Conv2D(n_filters, kernel_size=3, activation='...'))

# Si può anche usare una sintassi compatta passando la lista dei layer direttamente al costruttore
model = Sequential([
    Dense(n1, activation='...'),
    Dense(n2, activation='...')
])

# Compile, fit and evaluate a model
model.compile(loss='...', optimizer='...', metrics=['...'])
model.fit(X1, y1, epochs=2, batch_size=32, validation_split=0.2)
model.evaluate(X2, y2)
```

5.2 Regularizzazione L1 e dropout

- **Sintassi di riferimento:** `keras.regularizers.l1(lambda)` / `keras.layers.Dropout(rate)`

```
# Dropout
modello.add(Dropout(0.5))

# Layer with L1 regularization
```

```
modello.add(Dense(10, activation='...', kernel_regularizer=tf.keras.regularizers.l1(0.01),
bias_regularizer=tf.keras.regularizers.l1(0.01)))
```

5.3 Batch Normalization

Standardizza gli input del layer.

- **Sintassi di riferimento:** `keras.layers.BatchNormalization()`
- **Significato:** Normalizza l'output del layer precedente (media ≈ 0 , varianza ≈ 1) per ogni mini-batch.

```
from tensorflow.keras.layers import BatchNormalization
```

```
# Apply BatchNormalization
modello.add(BatchNormalization())
```

5.4 Early stopping

- **Sintassi di riferimento:** `keras.callbacks.EarlyStopping(monitor='val_loss', patience=n)`
- **Significato:** EarlyStopping monitora la metrica specificata sulla validation set e interrompe il training se non migliora per patience epoche consecutive.
restore_best_weights=True ripristina automaticamente i pesi dell'epoca migliore.
evaluate restituisce loss e metriche sul test set.

```
# Early stopping callback definition
# Monitor the specified loss and stop training if it doesn't improve for 2 consecutive epochs
ESCallback = tf.keras.callbacks.EarlyStopping(monitor='...', patience=2,
restore_best_weights=True)
```

```
# Define a Sequential model with Early Stopping
modello_ES = Sequential()
modello_ES.add(...)
```

```
modello_ES.compile(loss='...',
                    optimizer='...',
                    metrics=['...'])
```

```
# Train the model
history = modello_ES.fit(
    X1,
    y1,
    batch_size=64,           # Number of samples per gradient update
    epochs=...,              # Number of times to iterate over the training data
    validation_split=0.2,     # Proportion of training dataset for validation
    callbacks=[ESCallback]
)
```

Sezione 6: modelli ensemble e boosting (Scikit-learn e XGBoost)

6.1 Alberi di decisioni e random forest (Scikit-learn)

Modelli base e ensemble per classificazione.

- **Sintassi di riferimento:** `DecisionTreeClassifier(max_depth=n) / RandomForestClassifier(n_estimators=...)`
- **Significato:** `max_depth` limita la profondità dell'albero. `n_estimators` definisce il numero di alberi della foresta. Entrambi seguono l'interfaccia standard Scikit-Learn: `.fit(X_train, y_train) → .predict(X_test)`.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Albero di Decisione
dt_model = DecisionTreeClassifier(max_depth=..., random_state=0)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

# Random Forest
rf_model = RandomForestClassifier(n_estimators=..., random_state=0)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
```

6.2 Gradient boosting (Scikit-learn)

- **Sintassi di riferimento:** `GradientBoostingClassifier(n_estimators=..., learning_rate=..., max_depth=..., random_state=0)`
- **Significato:** `n_estimators` è il numero di weak learners. `learning_rate` scala il contributo di ogni albero.
- **Attenzione:** Se `y_pred` restituisce etichette stringa (object), convertire con `_, y_pred = np.unique(y_pred, return_inverse=True)` prima del calcolo delle metriche.

```
from sklearn.ensemble import GradientBoostingClassifier

# Gradient Boosting
gb_model = GradientBoostingClassifier(n_estimators=..., learning_rate=..., max_depth=...,
random_state=0)
gb_model.fit(X_train, y_train)
y_pred_gb = gb_model.predict(X_test)
_, y_pred_gb = np.unique(y_pred_gb, return_inverse=True) # se le label sono stringhe
```

6.3 XGBoost (XGBClassifier)

- **Sintassi di riferimento:** `XGBClassifier(n_estimators=..., learning_rate=..., random_state=0, device="cpu")`
- **Significato:** `.device="cpu"` forza il calcolo su processore, evitando problemi di allocazione GPU in laboratorio.

- **Attenzione:** Applicare il label Encoding **sia a y_train che a y_test** prima del fit: `_, y_train = np.unique(y_train, return_inverse=True)`.

```
# XGBoost (richiede Label numeriche)
%pip install xgboost
from xgboost import XGBClassifier
_, y_train_enc = np.unique(y_train, return_inverse=True)
_, y_test_enc = np.unique(y_test, return_inverse=True)
xgb_model = XGBClassifier(n_estimators=..., learning_rate=..., random_state=0,
device="cpu")
xgb_model.fit(X_train_enc, y_train_enc)
y_pred_xgb = xgb_model.predict(X_test_enc)
```

Sezione 7: altro

- Confusion matrix (sns.heatmap per visualizzare)


```
sklearn.metrics import confusion_matrix
confusion_matrix(array1, array2)
```
- `from sklearn.datasets import make_blobs`
Per generare dataset sintetico con cluster gaussiani. Utile per testare algoritmi di clustering.
- K-fold cross validation con sklearn


```
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=0)
for train_index, test_index in kf.split(X): # kf.split(X) returns the indices of the
training and test sets
```
- *# Ricavare gli elementi di una lista basandosi su elemento specifico*
`parametro_1, parametro_2 = max(lista, key=lambda x:x[0])` *# Guarda solo il primo parametro*
`lista_sorted = sorted(lista, key=lambda x:x[0])` *# Riordina guardando solo il primo parametro*
- Decision boundary con sklearn


```
from sklearn.linear_model import LogisticRegression
from utils import *
X_grid, xx, yy, X_2d = create_2d_meshpoints(X, resolution)
clf = LogisticRegression(multi_class='ovr', solver='lbfgs', max_iter=1000,
random_state=0)
clf.fit(X, y)
probability_function = clf.predict_proba
plot_decision_boundary_2d(X_grid, y, probability_function, xx, yy, X_2d,
n_features)
plot_probability_boundary(probability_function, X, y)
```